

* RCC - AHBIENB:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	OTGHS ULPIEN	OTGHS EN							DMA2 EN	DMA1 EN			BKP SRAMEN		
	YU	YU							YU	YU			YU		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			CRC EN					GPIOTH EN	GPIODG EN	GPIODF EN	GPIODE EN	GPIODP EN	GPIODC EN	GPIODB EN	GPIODA EN
			YU					YU	YU	YU	YU	YU	YU	YU	YU

Bit 7: GPIOTHEN: IO port H clock enable. This bit is set and cleared by software. 0 → IO port H clock disabled
1 → IO port H clock enabled

Bit 6: GPIODGEN: IO port G clock enable. This bit is set and cleared by software. 0 → IO port G clock disabled
1 → IO port G clock enabled

Bit 5: GPIODFEN: IO port F clock enable. This bit is set and cleared by software. 0 → IO port F clock disabled
1 → IO port F clock enabled

Bit 3: GPIODDEN: IO port D clock enable. This bit is set and cleared by software.

* GPIOx_ODR: (GPIO port output data register)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 → Reserved, must be kept at reset Value

Bits: 15:0 ODRy: port output data (y=0.....15)

These bits can be read and written by software

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIOx_ODR

* Bit 15:0 ODRy: port output data (y=0.....15)

* Direction Register: Direction Register is part of the GPIOx_MODER which stands for GPIO Mode Register.

GPIOx_MODER

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1,0]	MODER14[1,0]	MODER13[1,0]	MODER12[1,0]	MODER11[1,0]	MODER10[1,0]	MODER9[1,0]	MODER8[1,0]	MODER7[1,0]	MODER6[1,0]	MODER5[1,0]	MODER4[1,0]	MODER3[1,0]	MODER2[1,0]	MODER1[1,0]	MODER0[1,0]
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

Bits: 12:0 odd: bit output (0=0, 1=1)

Bits: 31:16 → bit output (0=0, 1=1)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1,0]	MODER6[1,0]	MODER5[1,0]	MODER4[1,0]	MODER3[1,0]	MODER2[1,0]	MODER1[1,0]	MODER0[1,0]	MODER7[1,0]	MODER6[1,0]	MODER5[1,0]	MODER4[1,0]	MODER3[1,0]	MODER2[1,0]	MODER1[1,0]	MODER0[1,0]
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

For each GPIO pin, there are two corresponding bits in the MODER Register. The value of two bits is as follows.

00 → Input

01 → output

* GPIOx_ODR: bit output (0=0, 1=1)

* ADC: 10k potentiometer Interfacing: Successive Approximation ADC and blinking LED by considering converted digital value which is stored in variable (ex: a)

program:

```
ADC_HandleTypeDef hadc1;
```

```
...
```

```
uint32_t a; // variable declaration
```

```
int main(void)
```

```
{
```

```
while(1)
```

```
{
```

```
    HAL_ADC_Start(&hadc1); // start ADC conversion
```

```
    if (HAL_ADC_PollForConversion(&hadc1, 1) == HAL_OK)
```

```
    {
```

```
        a = HAL_ADC_GetValue(&hadc1);
```

```
    }
```

```
    else
```

```
    {
```

```
        a = 0;
```

```
    }
```

// Blue LED will blink with a delay of 'a' variable value

```
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_SET);
```

```
    HAL_Delay(a);
```

```
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);
```

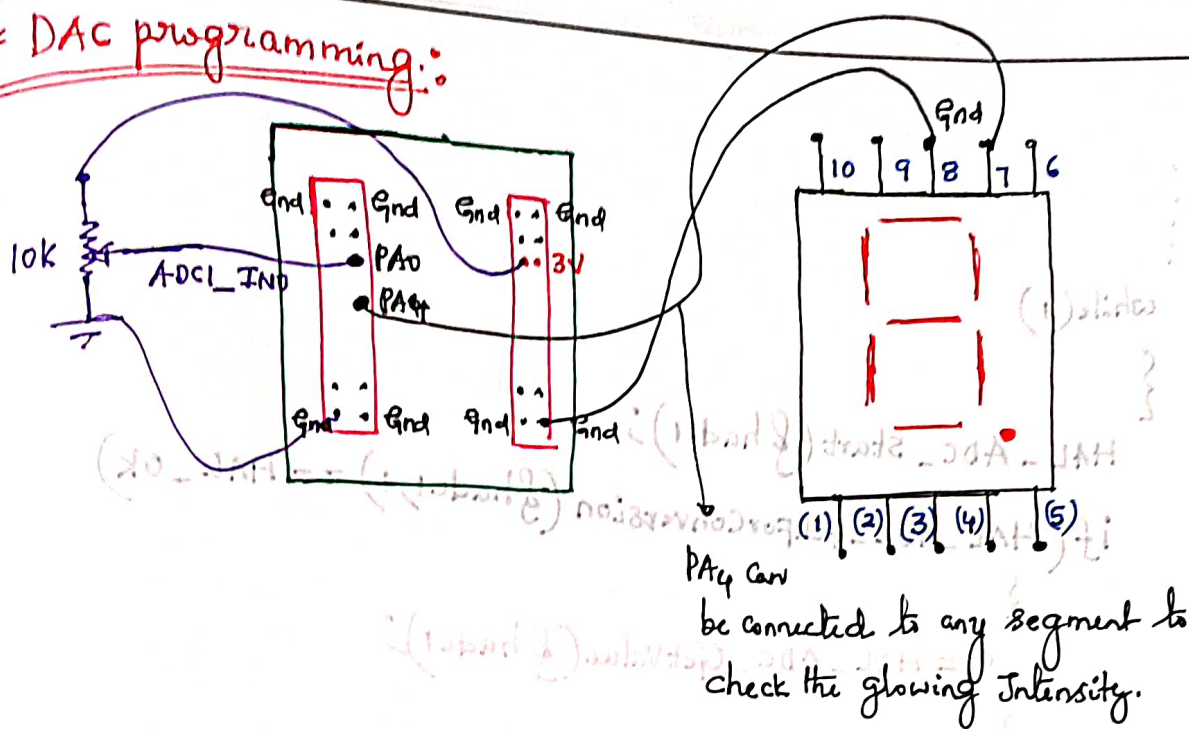
```
    HAL_Delay(a);
```

```
}
```

Setup pin configuration

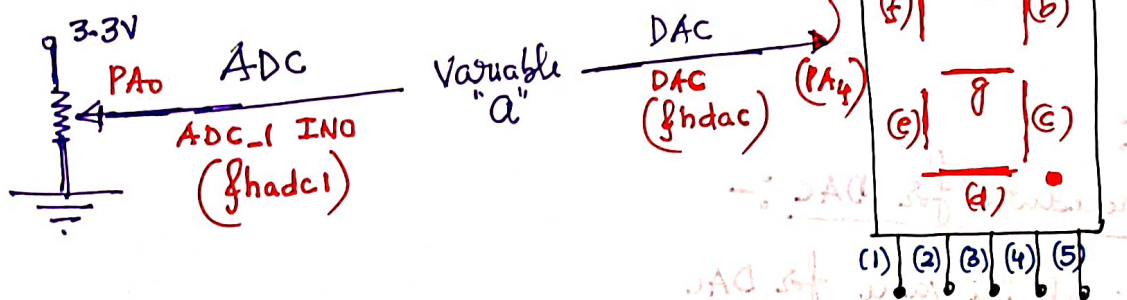
Setup pin PA0 to ADC1_IN0
PD15 to output

* DAC programming:



- 1) Setup Rcc ; Setup sys
- 2) Setup Analog: Setup ADC - select ADC1
ADC1 mode and configuration: Select IN0
By default configured PA4
- 3) Setup DAC: By default configured PA4
- 4) Setup project location & Name
- 5) Generate the code

* Summary:



Program:

```

ADC_HandleTypeDef hadc;
DAC_HandleTypeDef hdac;
uint32_t a;
...
int main (void)

```

```
while(1)
```

```
{
```

```
HAL_ADC_start(&hadc1);
```

```
if (HAL_ADC_PollForConversion(&hadc1, 1) == HAL_OK)
```

```
{
    a = HAL_ADC_GetValue(&hadc1);
```

```
else
```

```
{
    a = 0;
```

```
// First set the value for DAC
```

```
HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, a);
```

```
// Start DAC by selecting the channel
```

```
HAL_DAC_Start(&hdac, DAC_CHANNEL_1);
```

```
}
```

NOTE:

The procedure for DAC :-

→ First set the value for DAC

→ Need to pass DAC Handler and need to set the channel

→ Then data direction and data size and finally analog value